



Introduction to Artificial Intelligence
CSAI 350, Section 1, Fall 2024

8-Puzzle Solver Application with A* Search Algorithm and GUI

Hana Rahiman – 2022005663

Jana Shehata – 2022005444

Jannat Waqass – 2022005413

Lana Zanneh – 2022005620

Urita Sadallah – 2021004899

Date: 28th of November 2024

Instructor: Dr. Khaled Abdelgader Balawafi

Table of Contents

1- INTRODUCTION	3
2- IMPLEMENTATION	4
2.1- A* SEARCH ALGORITHM IMPLEMENTATION	4
2.2- GUI DESIGN AND DEVELOPMENT	5
2.3- ANIMATING THE PUZZLE-SOLVING PROCESS	7
3- CHALLENGES AND INSIGHTS	7
3.1- HANDLING UNSOLVABLE CONFIGURATIONS	7
3.2- ENSURING USER INPUT VALIDATION	8
3.3- VISUALIZING THE SOLVING PROCESS	8
4- RESULTS	9
5- CONCLUSION	12
6- TASK DISTRIBUTION	12

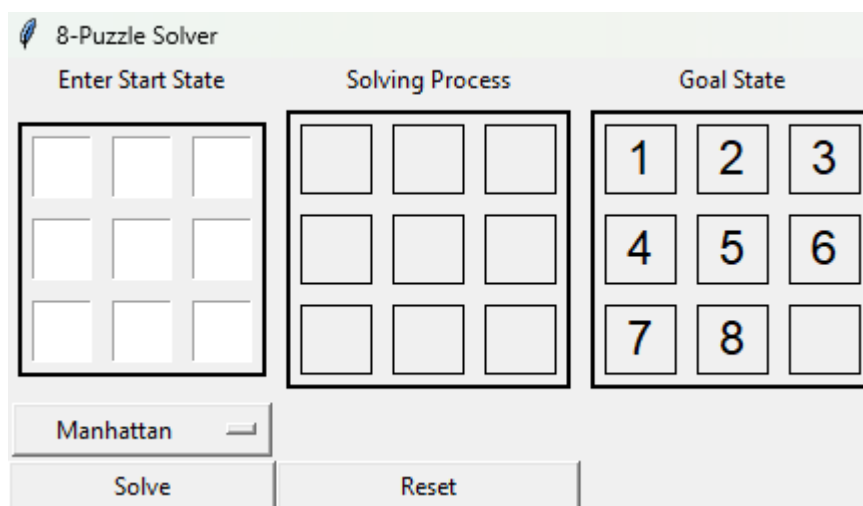
1- INTRODUCTION

Our goal for this project was to design and implement an 8-puzzle solver using the A* search algorithm, integrating two well-known heuristics: Manhattan distance and Hamming distance. To make the application more interactive and intuitive, we developed a graphical user interface (GUI) that allows users to input a start state, select a heuristic, and visualize the solving process dynamically as the puzzle progresses toward the goal state.

During the testing stage, we discovered an interesting mathematical property of the 8-puzzle that affects its solvability. Specifically, we learned that not all configurations of the puzzle can be solved due to the parity of inversions. After encountering unsolvable scenarios, ***we used ChatGPT, a generative AI tool, to deepen our understanding of this property.*** It helped us confirm that configurations with an odd inversion count are mathematically unsolvable.

To give readers a better understanding of our application, we have included a screenshot below. It showcases the main interface of the application, including the input grid, solving

grid, and goal state. This visual representation helps clarify the structure and functionality of our system.



(The “Manhattan” button is drop-down when we hover over it, allowing the user to choose which heuristic to use.)

2- IMPLEMENTATION

To develop our 8-puzzle solver, we focused on three main aspects: implementing the A* search algorithm, designing an intuitive graphical user interface (GUI), and animating the puzzle-solving process for enhanced user understanding. Below, we describe each component in detail and include relevant snippets with thorough explanations.

2.1- A* Search Algorithm Implementation

The A* algorithm combines cost-efficiency with informed heuristics to find the shortest path to the goal. We used Manhattan distance and Hamming distance as heuristics to guide the search process. Nodes represent puzzle states, and their **$f = g + h$** value determines the search priority.

Code: A* Algorithm with Heuristics

```
def manhattan_distance(state: List[List[int]]) -> int:
    """
    Calculate the Manhattan distance heuristic.
    This computes the sum of the distances of each tile from its goal position.
    """
    distance = 0
    for i in range(3): # Iterate through the rows
        for j in range(3): # Iterate through the columns
            if state[i][j] != 0: # Skip the blank tile
                x, y = divmod(state[i][j] - 1, 3) # Determine the goal position
                distance += abs(x - i) + abs(y - j) # Add the Manhattan distance
    return distance

def hamming_distance(state: List[List[int]]) -> int:
    """
    Calculate the Hamming distance heuristic.
    This counts the number of tiles that are not in their correct positions.
    """
    distance = 0
    for i in range(3): # Iterate through rows
        for j in range(3): # Iterate through columns
            if state[i][j] != 0 and state[i][j] != GOAL_STATE[i][j]: # Skip blank
                distance += 1 # Increment for misaligned tiles
    return distance

def a_star_search(start: List[List[int]], heuristic) -> List[Node]:
```

```

"""
Perform the A* search to find the optimal solution to the puzzle.
Parameters:
    - start: The initial puzzle configuration.
    - heuristic: The chosen heuristic function (Manhattan or Hamming).
Returns:
    - The solution path as a list of nodes, or None if unsolvable.
"""
open_set = [] # Priority queue to hold nodes to explore
start_node = Node(start, g=0, h=heuristic(start)) # Initial node with
heuristic
heapq.heappush(open_set, start_node) # Add the start node to the queue
closed_set = set() # Set to track visited states
closed_set.add(tuple(map(tuple, start))) # Add the initial state

while open_set: # While there are nodes to explore
    current = heapq.heappop(open_set) # Get the node with the lowest f value
    if current.state == GOAL_STATE: # If the goal state is reached
        path = [] # Reconstruct the solution path
        while current:
            path.append(current) # Add nodes to the path
            current = current.parent # Backtrack to the parent
        return path[::-1] # Return the path in correct order

    for neighbor in generate_moves(current): # Generate valid moves
        neighbor_tuple = tuple(map(tuple, neighbor.state))
        if neighbor_tuple in closed_set: # Skip already visited states
            continue
        neighbor.h = heuristic(neighbor.state) # Calculate heuristic for the
neighbor
        neighbor.f = neighbor.g + neighbor.h # Update f value
        heapq.heappush(open_set, neighbor) # Add neighbor to the queue
        closed_set.add(neighbor_tuple) # Mark neighbor as visited

return None # Return None if no solution is found

```

2.2- GUI Design and Development

We used Python's Tkinter library to build a user-friendly interface. The GUI allows users to input the initial state, select a heuristic, and reset the puzzle. The interface includes three grids: the input grid, the solving grid, and the goal grid.

Code: GUI Initialization

```

class PuzzleGUI:
    def __init__(self, root):
        """
        Initialize the GUI for the 8-puzzle solver.
        Parameters:
            - root: The main Tkinter root window.
        """
        self.root = root # Reference to the main window

```

```

        self.root.title("8-Puzzle Solver") # Set the title of the window
        self.start_state = [[0, 0, 0], [0, 0, 0], [0, 0, 0]] # Default blank
start state
        self.solution_path = None # Placeholder for the solution path

        # Frames for the three grids
        self.input_grid = [[None for _ in range(3)] for _ in range(3)] # Input
grid for start state
        self.solve_grid = [[None for _ in range(3)] for _ in range(3)] # Solving
process grid
        self.goal_grid = [[None for _ in range(3)] for _ in range(3)] # Goal
state grid

        # Add grid labels
        tk.Label(root, text="Enter Start State").grid(row=0, column=0) # Label
for input grid
        tk.Label(root, text="Solving Process").grid(row=0, column=1) # Label for
solving grid
        tk.Label(root, text="Goal State").grid(row=0, column=2) # Label for goal
grid

        # Create frames for each grid
        input_frame = tk.Frame(root, borderwidth=2, relief="solid")
        input_frame.grid(row=1, column=0, padx=5, pady=5)
        solve_frame = tk.Frame(root, borderwidth=2, relief="solid")
        solve_frame.grid(row=1, column=1, padx=5, pady=5)
        goal_frame = tk.Frame(root, borderwidth=2, relief="solid")
        goal_frame.grid(row=1, column=2, padx=5, pady=5)

        # Populate the grids with Tkinter widgets
        self.create_input_grid(self.input_grid, input_frame) # Create input grid
        self.create_grid(self.solve_grid, solve_frame) # Create solving grid
        self.create_grid(self.goal_grid, goal_frame) # Create goal state grid

        # Display the goal state in the goal grid
        self.display_state(self.goal_grid, GOAL_STATE)

        # Dropdown for heuristic selection
        self.heuristic_var = tk.StringVar(value="Manhattan")
        heuristic_menu = tk.OptionMenu(root, self.heuristic_var, "Manhattan",
"Hamming")
        heuristic_menu.grid(row=2, column=0, columnspan=1, sticky="ew") # Add
dropdown menu

        # Solve and reset buttons
        solve_button = tk.Button(root, text="Solve", command=self.solve_puzzle)
        solve_button.grid(row=4, column=0, columnspan=1, sticky="ew")
        reset_button = tk.Button(root, text="Reset", command=self.reset_puzzle)
        reset_button.grid(row=4, column=1, columnspan=1, sticky="ew")

```

2.3- Animating the Puzzle-Solving Process

To make the solving process interactive, we used an animation loop in which each step of the solution path updates the solving grid. The *update()* function ensures the interface refreshes in real time.

Code: Solution Animation

```
def animate_solution(self):
    """
    Animate the solving process on the solving grid.
    Displays each step in real-time with a brief delay.
    """
    for step, node in enumerate(self.solution_path, start=1):
        print(f"Step {step}:\n{node.state}\n")  # Print step details to the
console                                          # Update solving grid
        self.display_state(self.solve_grid, node.state)  # Refresh the GUI window
        self.root.update()  # Add a delay to visualize the animation
        time.sleep(0.5)
```

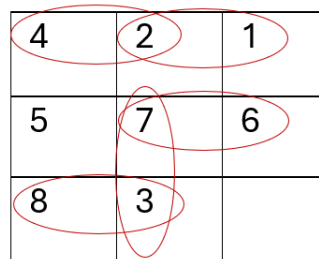
3- CHALLENGES AND INSIGHTS

In developing our 8-puzzle solver, we encountered several challenges that required critical thinking and research to resolve. These challenges not only deepened our understanding of the problem domain but also demonstrated the value of leveraging external tools, including generative AI, to support problem-solving. Below, we outline the main obstacles we faced and how we addressed them.

3.1- Handling Unsolvable Configurations

During testing, we discovered that certain initial puzzle states could not be solved due to their inversion parity. Specifically, an unsolvable puzzle has an odd inversion count, making it impossible to transition to the goal state. An inversion (or swap) is when a higher-numbered tile precedes a lower-numbered tile in reading order (left to right, top

to bottom) for the puzzle. The puzzle is solvable if and only if the number of inversions is even. For example:



4	2	1
5	7	6
8	3	

A puzzle with an arrangement like $[[4, 2, 1], [5, 7, 6], [8, 3, 0]]$ is unsolvable because it has an odd number of inversions (5).

If you input an unsolvable configuration, the algorithm will eventually exhaust all possible moves and return "No solution found" because it cannot reach the goal state.

- Additionally, we used ChatGPT to clarify the concept of inversions and validate our approach, ensuring it aligned with theoretical principles.

3.2- Ensuring User Input Validation

We noticed issues with incorrect or incomplete inputs causing errors during execution. For instance, users might forget to include all numbers from 0 to 8 or enter invalid characters.

- **Solution:** We added robust input validation in the GUI to ensure all required numbers are entered correctly and reject invalid inputs and mention that "No solution found". This improved both user experience and program stability.

3.3- Visualizing the Solving Process

Animating the solving process presented challenges related to real-time updates of the interface. Delays in updates or misaligned animations could confuse users.

- **Solution:** We implemented a smooth animation loop using ***time.sleep()*** and ***update()*** to dynamically refresh the solving grid after each step. This enhanced the clarity and engagement of the solving process.

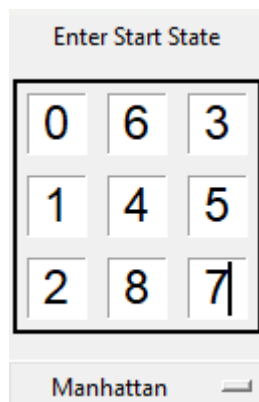
Through these challenges, we gained valuable insights into the complexities of the 8-puzzle problem and the importance of careful input handling, mathematical properties, and effective visualization techniques. Leveraging research and generative AI tools like ChatGPT proved instrumental in enhancing the robustness and functionality of our application.

4- RESULTS

The system successfully solves any valid and solvable 8-puzzle configuration.

Screenshots of the application interface and solving process are included below:

1. Initial State Input



Enter Start State		
0	6	3
1	4	5
2	8	7

Manhattan

2. Solving Process Animation

Step 1	Step 2	Step 3	Step 4																																				
<table><tr><td>1</td><td>6</td><td>3</td></tr><tr><td></td><td>4</td><td>5</td></tr><tr><td>2</td><td>8</td><td>7</td></tr></table>	1	6	3		4	5	2	8	7	<table><tr><td>1</td><td>6</td><td>3</td></tr><tr><td>4</td><td></td><td>5</td></tr><tr><td>2</td><td>8</td><td>7</td></tr></table>	1	6	3	4		5	2	8	7	<table><tr><td>1</td><td>6</td><td>3</td></tr><tr><td>4</td><td>8</td><td>5</td></tr><tr><td>2</td><td></td><td>7</td></tr></table>	1	6	3	4	8	5	2		7	<table><tr><td>1</td><td>6</td><td>3</td></tr><tr><td>4</td><td>8</td><td>5</td></tr><tr><td>2</td><td>7</td><td></td></tr></table>	1	6	3	4	8	5	2	7	
1	6	3																																					
	4	5																																					
2	8	7																																					
1	6	3																																					
4		5																																					
2	8	7																																					
1	6	3																																					
4	8	5																																					
2		7																																					
1	6	3																																					
4	8	5																																					
2	7																																						

Step 5		
1	6	3
4	8	
2	7	5

Step 6		
1	6	3
4		8
2	7	5

Step 7		
1		3
4	6	8
2	7	5

Step 8		
	1	3
4	6	8
2	7	5

Step 9		
4	1	3
	6	8
2	7	5

Step 10		
4	1	3
2	6	8
	7	5

Step 11		
4	1	3
2	6	8
7		5

Step 12		
4	1	3
2	6	8
7	5	

Step 13		
4	1	3
2	6	
7	5	8

Step 14		
4	1	3
2		6
7	5	8

Step 15		
4	1	3
	2	6
7	5	8

Step 16		
	1	3
4	2	6
7	5	8

Step 17		
1		3
4	2	6
7	5	8

Step 18		
1	2	3
4		6
7	5	8

Step 19		
1	2	3
4	5	6
7		8

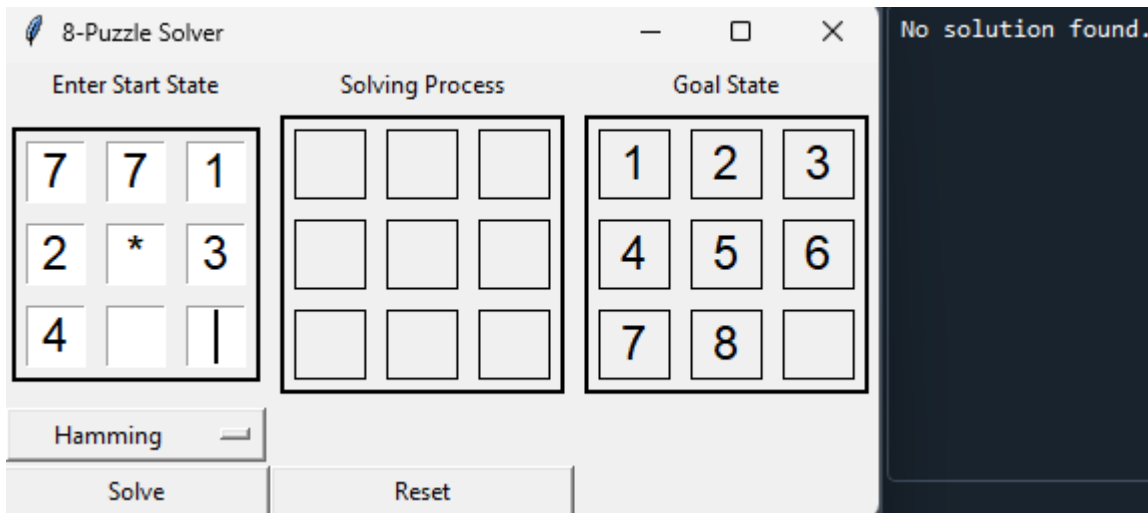
Step 20		
1	2	3
4	5	6
7	8	

3. Goal State Achieved

Solving Process			Goal State		
1	2	3	1	2	3
4	5	6	4	5	6
7	8		7	8	

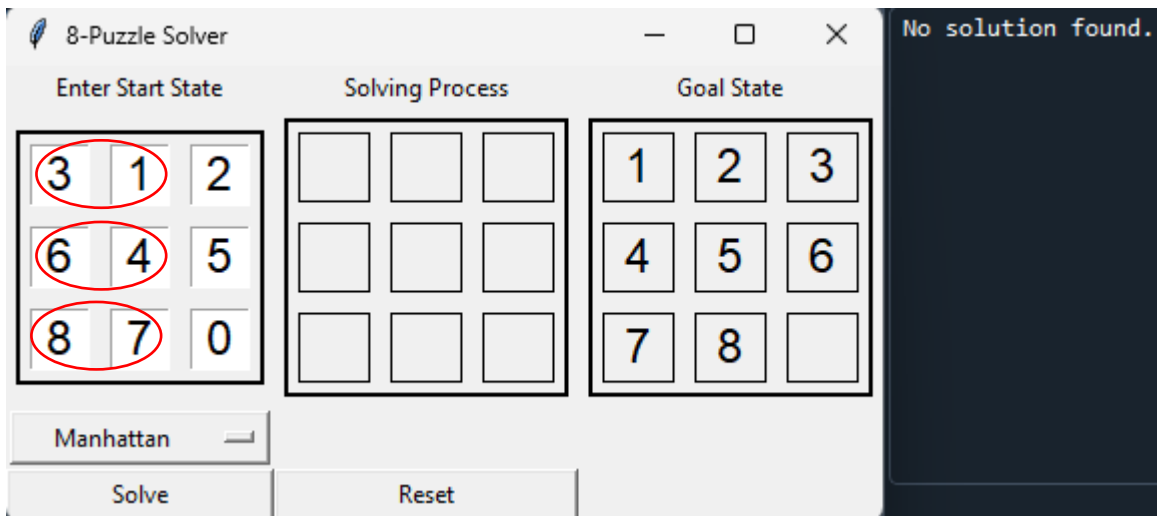
➤ **Invalid Puzzle Input:**

This configuration is invalid because it includes duplicate numbers, missing tiles, and special characters, violating the 8-puzzle rules. (Even if only one of the mentioned cases happen, it makes the input invalid.)



➤ **Unsolvable Due to Odd Inversion Parity:**

This puzzle cannot be solved because its inversion parity count is odd (3 as circled below), which makes it unsolvable by the 8-puzzle rules.



5- CONCLUSION

In conclusion, this project provided us with valuable hands-on experience in algorithm design, problem-solving, and GUI development. By implementing the A* search algorithm and utilizing heuristics like Manhattan and Hamming distance, we gained a deeper understanding of search techniques and their practical applications. Developing the GUI with Tkinter enhanced our ability to present complex algorithms in an accessible way; while overcoming challenges such as unsolvable puzzle configurations helped us appreciate the importance of rigorous testing and input validation. This project has not only sharpened our technical skills but also strengthened our problem-solving approach in real-world scenarios.

6- TASK DISTRIBUTION

Team Member	Role & Contributions
Hana Rahiman	- Designed an intuitive GUI to illustrate the starting and goal states of the puzzle. - Implemented animations to visualize the process of the A* algorithm, step by step, by dynamically updating tile movements.
Jana Shehata	- Assisted in analyzing and commenting on the code for every function. - Provided detailed explanations for each phase of the implementation, aiding in debugging.
Jannat Waqass	- Developed the GUI components of the project.

	- Discovered and researched the concept of "odd inversion count" for unsolvable configurations. - Provided examples and explanations of odd inversion counts and their impact on the puzzle. - Suggested adding extra cases in the results section of the report.
Lana Zanneh	- Contributed to the testing phase, identifying solvable and unsolvable configurations. - Assisted in testing input validation (e.g., ensuring valid number entries).
Urita Sadallah	- Implemented the A* search algorithm with Manhattan and Hamming heuristics. - Wrote the full report with all analysis of the different cases. - Documented and further commented on the full code.